

docs/handover.md - webRagSys 進捗管理

本ファイルは進捗管理用mdである。フェーズ完了ごとに更新する。形式：方針（計画）→実装（実行）→結果→残タスク→実行コマンド。人間とAIの双方がコンテキスト0%から再開できる粒度で相当詳しく記録する。

2026-09-19 初期化フェーズ（README + 進捗管理先行）

方針（計画）

- 対象フォルダ：webRagSys（親フォルダの兄弟フォルダ。他フォルダは読込不要）
- 要件定義書（20章）を受領済み。目的は社内文書CRUD + RAG + AIチャット。
- 学習目的の本質：Python → API → DB → LLM → RAG。
- 技術選定の確定事項：
 - LLM／Embedding：OpenAI利用（本番）。既定は課金回避の偽実装
 - RAGフレームワーク：LangChain固定
 - VectorDB：pgvector（PostgreSQL拡張で一元管理）
 - DB方針：PostgreSQL単一
 - 対象者：Python中級者、RAG初心者向けにbeginner_overview.mdを作成
- 順序の指定：
 - docs（beginner_overview等）とdiagramsはテストの後に作成し、PDF生成する
 - READMEと進捗管理用mdは先行作成する
- AGENTS.mdは親フォルダのマスタAGENTS.mdを継承し、内容に従う
- 参考資料：
 - stockChecker/docs/beginner_overview.md（構成見本）
 - stockChecker/diagrams（01_flowchart、02_sequence、03a、03b、04_mindmap、05_state、.mmd + .md + .pdf、convert_to_pdf.ps1 + puppeteer設定）
 - stockChecker/README.md（README構成見本）
 - webScraping/docs/handover.md（進捗管理の記録粒度見本）
- PDF方針：
 - 図PDFはMermaid CLI（npx @mermaid-js/mermaid-cli、--pdfFit、03aのみ横A4）で生成
 - docs PDFはMarkdown→PDFで生成（手段は実装確定後に再確認）
- セキュリティ：ローカルパス、APIキー、個人ID、認証情報は一切記録しない。機密情報は.env（Git除外）のみ。

実装（実行）

- webRagSysフォルダの空を確認（Get-ChildItemで0件）
- 環境確認：Python 3.11.15、pip 26.1.2、git 2.54.0、Docker未検出（CommandNotFound）
- webRagSys/docsフォルダ作成
- webRagSys/AGENTS.md作成：
 - 親マスタ02テンプレート継承（概要、環境、禁止情報、GitHub運用、分割コミット、handover必須更新）
 - webRagSys固有化（FastAPI、PostgreSQL + pgvector、LangChain、偽実装既定、ディレクトリ構造、RAG注意点）
 - 規約：先頭コメント7項目、config集約、main／validate／execute／format分離、単一責務、型ヒント必須、PEP8、例外明示
 - docs／diagramsはテスト後 + PDF化、README／handoverは先行 + 随時更新を明記
- webRagSys/README.md雛形作成（要件定義書14章準拠）：
 - 概要、機能、技術表、構成ツリー、セットアップ、起動、API一覧6件、RAG概要図、使用例curl、テスト、ドキュメント案内、拡張案、完成条件、注意点
 - .env.example運用、偽実装既定を明記

- 本handover.md作成（本ファイル）

結果

- 3ファイル生成完了：
 - webRagSys/AGENTS.md
 - webRagSys/README.md
 - webRagSys/docs/handover.md
- 単体テスト：未実施（実装なしのため）
- Lint：未実施（実装なしのため）
- PDF：未生成（テスト後に実施する指定のため）
- 発見事項：
 - Docker未導入のため、完成条件「Dockerで環境構築」は後工程でブロッカーになる可能性あり
 - Python実測3.11のため、AGENTS.mdでは3.10+互換として記載
 - RUG表記はRAGとして扱う

残タスク

- ☐ 骨格実装：config.py、requirements.txt、.env.example、.gitignore、Dockerfile、docker-compose.yml、src雛形
- ☐ 文書CRUD実装：schemas、models、repository、routers/documents
- ☐ RAG実装：chunker、embeddings（本番／Fake切替）、vectorstore（pgvector）、chain（LangChain）
- ☐ chat実装：routers/chat（検索→プロンプト組立→LLM呼出→回答＋参照ID）
- ☐ LLM偽実装：provider切替（USE_FAKE既定true）
- ☐ テスト実装＋実行：pytest全件成功、ruffエラー0（docs着手条件）
- ☐ docs本格作成（テスト後）：beginner_overview.md（Python中級・RAG初心者向け、LangChain中心）、README追記
- ☐ diagrams作成（テスト後）：01_flowchart、02_sequence、03a_class_rag、03b_class_modules、04_mindmap、05_state（.mmd＋.md）
- ☐ PDF生成：docs PDF、diagrams PDF（Mermaid CLI＋puppeteer設定複製）
- ☐ Docker検証：PostgreSQL＋pgvector起動、API接続
- ☐ GitHub公開：public、分割コミット、push前禁止情報チェック

実行コマンド

```
# 環境確認（実施済み）
python --version
pip --version
git --version

# フォルダ確認（実施済み）
Get-ChildItem -Force -LiteralPath "webRagSys"

# 今後のテスト／Lint（予定）
python -m pytest tests -q
python -m ruff check src tests config.py

# 今後の起動（予定）
uvicorn src.api.main:app --reload
```

```
# 今後の図PDF変換（予定、Node.js前提）
# npx -y @mermaid-js/mermaid-cli -i diagrams/01_flowchart.mmd -o diagrams/01_flowchart.pdf --pdfFit
```

実行成果物

- webRagSys/AGENTS.md
- webRagSys/README.md
- webRagSys/docs/handover.md

公開禁止情報

公開禁止情報は含まれていません

2026-09-19 実装フェーズ（骨格＋CRUD＋RAG＋chat＋テスト）

方針（計画）

- 先行作成済みのAGENTS.md、README.md雛形に従い、最小構成で完成条件1～8と11を満たす
- LangChain固定、既定USE_FAKE=trueで課金なし。EmbeddingはFakeベクトル、LLMはFake定型文
- DBは本番PostgreSQL＋pgvector、テストとドライバ欠落時はSQLite退避で動作保証する
- docsとdiagramsはテストの後に作成する指定のため、本フェーズでは作らない

実装（実行）

- 骨格：requirements.txt、config.py（Settings集約）、.env.example、.gitignore、Dockerfile、docker-compose.yml（pgvector/pg16）、pyproject.toml（ruff設定）
- DB：src/db/database.py（engine遅延生成、Postgres拡張、get_db、init_db）、src/db/models.py（documents、document_chunks、Postgres時Vector／他時JSON）
- スキーマ：src/schemas/document.py、src/schemas/chat.py
- RAG：src/rag/chunker.py（LangChain優先＋単純分割退避）、src/rag/embeddings.py（Fake既定／OpenAI本番）、src/rag/vectorstore.py（save／delete／cosine検索）、src/rag/chain.py（answer_question）
- LLM：src/llm/fake_provider.py、src/llm/provider.py（USE_FAKE切替）
- API：src/api/main.py（create_app、health、startup）、src/api/routers/documents.py（CRUD5件、404／422処理、登録更新時にchunks再生成）、src/api/routers/chat.py（POST /chat、500処理）
- テスト：tests/conftest.py（SQLite＋dependency override）、tests/test_documents.py（5件）、tests/test_chat.py（RAG利用＋chunker）
- 修正：
 - 初回pytestでpsycopg2欠落エラー→database.pyを遅延生成＋SQLite退避に修正
 - ruffでUP009／S101／B008／E501等→pyprojectでUP009とB008除外、testsのS101除外、行長100、E501の3か所改行、main.pyのbroad-exceptをOSError／RuntimeError＋loggingに修正

結果

- 単体テスト：7件すべて成功（python -m pytest tests -q）
- Lint：ruffエラー0（python -m ruff check src tests config.py → All checks passed）
- 検証：POST /documents→GET／PUT／DELETE、存在しないIDで404、POST /chatでFAKE回答＋source_ids取得を確認
- PDF：未生成（テスト後工程のため）

残タスク

- ☐ docs本格作成（テスト後）：beginner_overview.md（Python中級・RAG初心者向け）、README追記

- ☐ diagrams作成（テスト後）：01_flowchart、02_sequence、03a_class_rag、03b_class_modules、04_mindmap、05_state（.mmd + .md）
- ☐ PDF生成：docs PDF、diagrams PDF（Mermaid CLI + puppeteer設定）
- ☐ Docker検証：Docker導入後にcompose起動とPostgreSQL + pgvector接続確認
- ☐ GitHub公開：public、分割コミット、push前禁止情報チェック

実行コマンド

```
# 依存
pip install -r requirements.txt
# テスト/Lint
python -m pytest tests -q
python -m ruff check src tests config.py
# 起動
uvicorn src.api.main:app --reload
```

実行成果物

- config.py、requirements.txt、pyproject.toml、Dockerfile、docker-compose.yml、.env.example、.gitignore
- src/api/schemas/db/rag/llm/utils一式
- tests/conftest.py、test_documents.py、test_chat.py

公開禁止情報

公開禁止情報は含まれていません

2026-09-19 ドキュメント・図フェーズ（テスト後）

方針（計画）

- 指定順序どおりテストの後にdocsとdiagramsを作成し、両方PDF化する
- beginner_overview.mdはstockChecker構成を踏襲し、対象をPython中級・RAG初心者に切替える
- RAGフレームワークはLangChain固定、偽実装で課金を避ける
- 図はstockCheckerと同形式の6種（.mmd + .md + .pdf）

実装（実行）

- docs/beginner_overview.md作成（実コード照合済み）：
 - 1章：構成と役割表、2章：API→RAG→DB/LLM依存図、3章：登録→RAG化→質問応答フロー
 - 4章：Depends、from_attributes、TextSplitter、Fake切替、pgvector、コサイン、.env区別
 - 付録：STEP1-12学習順
- diagrams 6種作成（.mmd + .md）：
 - 01_flowchart：起動→CRUD分岐→RAG化→chat分岐
 - 02_sequence：登録と質問応答の通信
 - 03a_class_rag：Chunker/Embeddings/VectorStore/Chain/LLM
 - 03b_class_modules：Main/Routers/DB/Settings
 - 04_mindmap：全体概念階層
 - 05_state：PENDING→CHUNKING→EMBEDDING→SAVING→COMPLETED
- diagrams/convert_to_pdf.ps1 + puppeteer設定2件をstockCheckerから流用（03aのみ横A4）
- PDF生成：図6件はmermaid-cli、docs 3件（beginner_overview、handover、README）はmd-to-pdf
- README追記：ドキュメント節を本格版に更新

- 不具合と修正：
 - convert_to_pdf.ps1がPS5.1で解析エラー→原因は矢印文字のUTF-8 BOMなし問題、英字表記に書換えて解決、直接実行で6件変換を確認
 - md-to-pdfのpdf-options指定がPowerShell引用で失敗→既定オプションで生成に変更
 - pytest間欠失敗（no such table: documents）→原因はdb fixture実行時点でmodels未importのためcreate_allが空振りし、旧test.db残存時のみ偶然成功していた。conftestにmodels import追加 + tmp_path毎テスト分離で解決、3連続7件成功を確認

結果

- 単体テスト：7件すべて成功（3連続確認）
- Lint：ruffエラー0
- PDF：図6件 + docs 3件の計9件を生成
- 図PDF：01_flowchart、02_sequence、03a_class_rag、03b_class_modules、04_mindmap、05_state
- docs PDF：beginner_overview.pdf、handover.pdf、README.pdf

残タスク

- ☐ Docker検証：Docker導入後にcompose起動とPostgreSQL + pgvector接続確認
- ☒ GitHub公開：public、分割コミット、push前禁止情報チェック

実行コマンド

```
# テスト/Lint
python -m pytest tests -q
python -m ruff check src tests config.py
# 図PDF再生成
./diagrams/convert_to_pdf.ps1
# docs PDF再生成
npx md-to-pdf docs/beginner_overview.md
npx md-to-pdf docs/handover.md
npx md-to-pdf README.md
# 起動
uvicorn src.api.main:app --reload
```

実行成果物

- docs/beginner_overview.md + .pdf
- docs/handover.md + .pdf（本ファイル）
- README.md + README.pdf（追記済み）
- diagrams 6種（.mmd + .md + .pdf）、convert_to_pdf.ps1、puppeteer設定2件

公開禁止情報

公開禁止情報は含まれていません

2026-09-19 GitHub公開フェーズ

方針（計画）

- publicリポジトリで公開し、履歴が読める分割コミットにする
- push前に公開禁止情報（.env、実パス、キー）をチェックする

- Dockerは未導入のため検証は保留し、導入後の手順を残す

実装（実行）

- 状態確認：Docker未検出、gh 2.96.0認証済み（HTTPS）、git未初期化を確認
- git init -b main、user.nameとuser.emailをリポジトリローカルに設定（noreply形式）
- 分割コミット6件：
 - 骨格・設定・Docker基盤
 - DB・スキーマ・RAG・LLM偽実装
 - API（documents CRUD・chat）
 - テスト（CRUD・chat偽実装）
 - docs・図・PDF
 - srcパッケージ初期化ファイル
- push前チェック：.env未追跡、実パスなし、キーなしを確認
- gh repo create webRagSys --public --source . --remote originで作成
- git push -u origin mainで公開

結果

- 公開先：<https://github.com/t-tani-it/webRagSys>
- mainがorigin/mainを追跡する状態を確認
- git statusはクリーン（handover更新分を除く）

残タスク

- ☐ Docker検証：Docker Desktop導入後にdocker compose up --buildとPostgreSQL + pgvector接続確認
- ☐ 本フェーズのhandover更新分をコミット + pushする

実行コマンド

```
git log --oneline
git push -u origin main
```

公開禁止情報

公開禁止情報は含まれていません